

API Specification (v1)

The SALESTORM API is versioned under `/api/v1` and follows RESTful conventions. APIs use JSON, JWT-based authentication, correlation IDs for end-to-end tracing, and idempotency controls for critical business operations.

1. Common API Standards

- **Authentication:** Authorization: Bearer <JWT>
- **Correlation:** X-Correlation-Id; generated by the API Gateway when absent
- **Content Type:** application/json
- **Idempotency:** Idempotency-Key for reservation, purchase, and payment operations
- **Rate Limiting:** 429 Too Many Requests with Retry-After
- **API Versioning:** /api/v1

Standard Error Response

```
{
  "error": {
    "code": "OUT_OF_STOCK",
    "message": "Product is sold out",
    "correlationId": "c-123"
  }
}
```

2. Product APIs

GET `/api/v1/products/{id}`

Returns product information, pricing, and sale-window details.

Success — 200 OK

```
{
  "id": "P1001",
  "name": "Product Name",
  "price": 4999,
  "salePrice": 2999,
  "saleWindow": {
```

```
"start": "2026-10-10T10:00:00Z",
"end": "2026-10-10T10:30:00Z"
}
}
```

Errors: 404 Not Found

Characteristics: Read-only and CDN-cacheable.

GET /api/v1/products/{id}/inventory

Returns the currently displayed inventory quantity.

Success — 200 OK

```
{
  "available": 37,
  "total": 100
}
```

Errors: 404 Not Found

The returned quantity is for display only and may be cached. The Inventory Service performs the authoritative stock check during reservation.

3. Cart APIs

POST /api/v1/cart/items

Adds or updates a product in the customer's cart.

Request

```
{
  "productId": "P1001",
  "quantity": 1
}
```

Success — 201 Created

```
{
```

```
"cartId": "C10001",
"items": [
  {
    "productId": "P1001",
    "quantity": 1
  }
]
```

Errors: 400 Bad Request, 401 Unauthorized

The same product within a cart is handled as an upsert. Cart operations do not reserve inventory.

DELETE /api/v1/cart/items/{id}

Removes an item from the cart.

Success: 204 No Content

Errors: 404 Not Found

The operation is safe to repeat.

4. Reservation API

POST /api/v1/reservations

Creates a temporary reservation against authoritative inventory.

Request

```
{
  "productId": "P1001",
  "quantity": 1
}
```

Header

Idempotency-Key: <unique-key>

Success — 201 Created

```
{
  "reservationId": "R10001",
  "status": "RESERVED",
  "expiresAt": "2026-10-10T10:10:00Z"
}
```

Errors

- 409 OUT_OF_STOCK
- 422 IDEMPOTENCY_KEY_CONFLICT
- 429 TOO_MANY_REQUESTS
- 503 SERVICE_UNAVAILABLE

A repeated request with the same idempotency key returns the previously stored result and does not create another reservation.

5. Purchase API

POST /api/v1/purchases

Provides the primary purchase entry point by coordinating inventory reservation and payment initiation.

Request

```
{
  "productId": "P1001",
  "quantity": 1,
  "paymentToken": "tok_xxx"
}
```

Header

Idempotency-Key: <unique-key>

Success — 201 Created

```
{
  "reservationId": "R10001",
  "paymentStatus": "PENDING"
}
```

Errors: 409, 422, 429, 503

A retry using the same idempotency key returns the stored response. No second reservation or payment is created.

6. Payment API

POST /api/v1/payments

Initiates payment for a valid inventory reservation.

Request

```
{
  "reservationId": "R10001",
  "paymentToken": "tok_xxx"
}
```

Header

Idempotency-Key: <unique-key>

Success — 202 Accepted

```
{
  "paymentId": "PAY10001",
  "status": "PENDING"
}
```

A payment completed synchronously may return 200 OK with status: SUCCEEDED.

Errors

- 402 PAYMENT_FAILED
- 404 NOT_FOUND
- 409 RESERVATION_EXPIRED
- 504 PAYMENT_TIMEOUT

A payment timeout does not imply failure. The Payment Service moves the transaction to UNKNOWN and reconciles its status with the payment provider.

Only one logical payment is permitted for a reservation.

7. Order API

GET /api/v1/orders/{orderId}

Returns the current state of an order.

Success — 200 OK

```
{
  "orderId": "ORD10001",
  "status": "CONFIRMED",
  "paymentId": "PAY10001"
}
```

Errors: 404 Not Found

Order creation occurs asynchronously after successful payment. Therefore, a temporary 404 is possible while the PaymentSucceeded event is waiting to be processed.

Characteristics: Read-only and safe to retry.

8. Admin Simulation API

GET /api/v1/admin/simulation

Returns metrics from the flash-sale simulation and system execution.

Required Role: ADMIN

Success — 200 OK

```
{
  "metrics": {
    "requestsPerSecond": 12000,
    "reservationsAttempted": 10000,
    "reservationsSuccessful": 100,
    "paymentsSucceeded": 95,
    "paymentsFailed": 5,
    "duplicateRequests": 200,
    "ordersCreated": 95
  }
}
```

Errors: 403 Forbidden

The endpoint is read-only.

9. HTTP Status Semantics

409 Conflict

Used for expected business conflicts such as OUT_OF_STOCK or an expired reservation.

422 Unprocessable Entity

Used when an idempotency key is reused with a different request payload.

429 Too Many Requests

Indicates rate or concurrency limiting. The response includes Retry-After.

503 Service Unavailable

Returned when a critical dependency such as the authoritative inventory database is unavailable. The system fails closed rather than estimating or guessing stock.

504 Gateway Timeout

Indicates a dependency timeout. For payment operations, the state becomes UNKNOWN and is reconciled instead of being marked as failed.

10. Event Contracts

SALESTORM uses versioned JSON events over a durable message broker. Events are keyed by reservationId to maintain ordering for a purchase flow.

Reservation Events

ReservationCreated / ReservationExpired

```
{
  "reservationId": "R10001",
  "productId": "P1001",
  "customerId": "C10001",
  "expiresAt": "2026-10-10T10:10:00Z"
}
```

Payment Events

PaymentInitiated

```
{  
  "paymentId": "PAY10001",  
  "reservationId": "R10001",  
  "amount": 2999  
}
```

PaymentSucceeded

```
{  
  "paymentId": "PAY10001",  
  "reservationId": "R10001",  
  "customerId": "C10001",  
  "amount": 2999,  
  "providerRef": "PG-78901"  
}
```

PaymentFailed

```
{  
  "paymentId": "PAY10001",  
  "reservationId": "R10001",  
  "reason": "PAYMENT_DECLINED"  
}
```

Order Event

OrderCreated

```
{  
  "orderId": "ORD10001",  
  "paymentId": "PAY10001",  
  "items": [  
    {  
      "productId": "P1001",  
      "quantity": 1  
    }  
  ],  
  "total": 2999  
}
```

```
}
```

Fulfilment and Notification Events

ShipmentUpdated

```
{  
  "orderId": "ORD10001",  
  "status": "SHIPPED"  
}
```

NotificationRequested

```
{  
  "orderId": "ORD10001",  
  "channel": "EMAIL",  
  "template": "ORDER_CONFIRMED"  
}
```

11. Event Reliability

- Events use **at-least-once delivery**.
- Consumers are **idempotent** and safely handle duplicate events.
- Failed messages are retried using bounded exponential backoff.
- Repeatedly failed messages are moved to a **Dead Letter Queue (DLQ)**.
- Event schemas are versioned to support controlled evolution.
- PaymentSucceeded is persisted reliably before publication through the **Transactional Outbox** pattern.
- The durable broker retains events while the Order Service is unavailable and delivers them after recovery.
- `eventId` and `correlationId` provide traceability across asynchronous workflows.